NETWORKING FOR DEVOPS

Practical Workbook

From Fundamentals to Secure Services

12 labs · 38 tasks · Two exercise sets, one English learning path

Companion to the chapter by Mohammad Fatahi

English translation and consolidation of the supplied networking exercises, with clearly separated editorial notes.

This English workbook combines both supplied networking assignments into one learning sequence: **12 labs and 38 tasks**. Task requirements, expected filenames, hints, alternatives, and original point values are retained. Labs 1-5 come from Network Assignment 01; Labs 6-12 come from Network Assignment 02. New task numbers follow the lab number: for example, Task 6.1 was Assignment 02, Task 1.1.

You are a junior DevOps engineer at a technology company. You need to prepare a Linux host for a web application, configure network services, investigate failures, and document what you observe. Work on a lab host or localhost where appropriate. Save commands, output, and explanations separately; distinguish executed results from theoretical answers or simulations.

Notes about the source exercises

• Assignment 01 states **80 points**, plus a **10-point bonus** scenario. Its required scenario scores sum to 80.

• Assignment 02 states **60 points** in its introduction, while the listed scenario/task scores sum to **70**. Both values are recorded here; no grading total has been silently changed.

• The original packet-capture requirement is 30 seconds, but its example uses packet limits. A packet limit and a duration are different; see the editorial note in Lab 2.

• The FTP question requests anonymous access while its hint disables it. The discrepancy is preserved and explained in Lab 7.

• Lab 11 asks for the **10 most recent events**, not events from the last ten minutes.

• Historical submission deadlines and course-panel instructions appear in the appendix. They are not a new deadline for this public workbook.

Lab 1 - Preparing a Web Server (20 points)

Prepare a Linux server to host the company's web application.

Task 1.1 - Basic Network Information (5 points)

Write commands that display all network interfaces, the current IP address, the default gateway, and the configured DNS servers. Save these details in `network_info.txt`.

Hints: Use `ip addr`, `ip route`, and `cat /etc/resolv.conf`. You can also inspect interfaces with `ifconfig` or `ip link`.

Task 1.2 - A Static IP Address with Netplan (5 points)

Create `01-netcfg.yaml` with the following configuration for the primary interface:

• Static address: `192.168.1.100/24`.

• Gateway: `192.168.1.1`.

• DNS servers: `8.8.8.8` and `1.1.1.1`.

Hints: Netplan files normally live in `/etc/netplan/`. YAML indentation matters. Find the real interface name using `ip link`; examples include `eth0`, `ens33`, and `enp0s3`.

Task 1.3 - Connectivity Tests (5 points)

Write `test_connectivity.sh` to send five ping packets to `8.8.8.8`, send five packets to `google.com`, test DNS resolution for `github.com`, and save the results in `connectivity_report.txt`.

Hints: Use `ping -c 5` and `nslookup` or `dig`. Append output with >>.

Task 1.4 - Ports and Connections (5 points)

Display all listening TCP ports, current established connections, and the processes listening on ports 80 and 443. Save the output in `ports_status.txt`.

Hints: Use `netstat -tuln` or `ss -tuln`. For processes, use `netstat -tulnp` or `ss -tulnp`; sudo may be required. Use `grep` to filter output. The `-u` examples include UDP as well as TCP; select the transport relevant to the task.

Lab 2 - Troubleshooting Network Problems (25 points)

Your production server has a problem: users cannot access the website. Reproduce tests in a suitable lab rather than changing a production host without a plan.

Task 2.1 - Traffic Analysis with tcpdump (8 points)

Capture HTTP traffic on port 80 for 30 seconds. Capture DNS traffic separately. Save `http_capture.pcap` and `dns_capture.pcap`, then describe your observations in `tcpdump_analysis.txt`.

Original example commands:

```
sudo tcpdump -i [interface] port 80 -w http_capture.pcap -c 100
sudo tcpdump -i [interface] port 53 -w dns_capture.pcap -c 50
```

Hints: Find the interface using `ip link`. Read a capture with `tcpdump -r http_capture.pcap`. The original hint gives this more specific IPv4/TCP payload filter:

```
sudo tcpdump -i eth0 'tcp port 80 and (((ip[2:2] - ((ip[0]&0xf)<<2)) - ((tcp[12]&0xf0)>>2)) != 0)'
```

Editorial note: Replace `[interface]` with an actual name. `-c 100` stops after 100 packets, not 30 seconds. To satisfy the stated duration, use a timed capture such as `sudo timeout 30 tcpdump -i eth0 port 80 -w http_capture.pcap`. Generate relevant traffic during the capture. Keep raw captures private when they contain credentials, browsing activity, or unrelated host data.

Task 2.2 - Wireshark Analysis (7 points)

Use the captures from Task 2.1 to answer:

1. How many TCP connections were established?

2. How many DNS queries were captured, and for which domains?

3. Is there evidence of packet loss? Inspect TCP retransmissions.

4. What is the average response round-trip time (RTT)?

Save answers in `wireshark_analysis.txt` and include relevant Wireshark screenshots.

Hints: Open `wireshark http_capture.pcap`. Use Statistics > Conversations > TCP. Filter retransmissions with `tcp.analysis.retransmission`. If the GUI is unavailable, use `tshark`.

Editorial note: Capture gaps and reordering can also affect retransmission analysis. Explain how you measured RTT and how many samples support the average.

Task 2.3 - Connection Tests with Telnet (5 points)

Write `port_check.sh` to test and report whether the web server on port 80, SSH on port 22, and MySQL on port 3306 are reachable from a client outside the service host.

Hints: `telnet localhost 80` checks a local port. `nc -zv localhost 80` is an alternative. Use `timeout 3 telnet localhost 3306` to limit waiting. Replace localhost with the actual service address for the external-client requirement.

Task 2.4 - The OSI Model and Troubleshooting (5 points)

Create `osi_troubleshooting.md`. Briefly explain each of the seven OSI layers in one or two lines. For a user who cannot access a website, describe the checks from Layer 1 through Layer 7 and give at least one Linux tool or command for each layer.

Hints: Physical: cable, link, LEDs. Data Link: MAC address, switch, ARP. Network: IP, routing, ping. Transport: TCP/UDP, ports, netstat. Some session and presentation functions are implemented within the application stack; explain what your proposed test actually observes.

Lab 3 - Network Security (20 points)

The company wants to improve server security.

Task 3.1 - Firewall Configuration with UFW or iptables (10 points)

Write `setup_firewall.sh` to enable the firewall, allow SSH on port 22 only from the local address/network, allow HTTP on port 80 and HTTPS on port 443 from anywhere, block other incoming traffic, allow outgoing traffic, and save the active rules in `firewall_rules.txt`.

Original UFW hints:

```
sudo ufw enable
sudo ufw allow 80/tcp
sudo ufw allow from 192.168.1.0/24 to any port 22
sudo ufw status numbered > firewall_rules.txt
```

Original iptables hints:

```
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
sudo iptables -A INPUT -p tcp -s 192.168.1.0/24 --dport 22 -j ACCEPT
sudo iptables -L -n > firewall_rules.txt
```

Editorial note: These are partial hints, not a complete policy. Preserve your actual administrative access before enabling restrictions. Existing broad SSH rules can defeat the intended restriction. Review rule ordering and established-connection handling when using iptables.

Task 3.2 - Monitoring Suspicious Traffic (10 points)

Write `monitor_suspicious.sh` to identify addresses with more than 100 connections, extract failed SSH logins, list addresses communicating with unusual ports above 10000, and save a full report in `security_report.txt`.

Hints:

```
netstat -ntu | awk '{print $5}' | cut -d: -f1 | sort | uniq -c | sort -n
sudo grep "Failed password" /var/log/auth.log
```

Use `wc -l` to count lines. Authentication logs may be in `/var/log/auth.log` or `/var/log/secure`.

Editorial note: Define whether a port is a local service port or a remote client port. High ephemeral client ports are normal. A connection-count threshold is a reason to investigate, not proof of an attack. The simple parsing hint needs adaptation for IPv6.

Lab 4 - DNS, Routing, and Topology (15 points)

Your team wants internal name resolution and a documented network design.

Task 4.1 - DNS Resolution Configuration (5 points)

Add these entries to `/etc/hosts`:

```
192.168.1.10 app.local
192.168.1.11 db.local
192.168.1.12 cache.local
```

Change the DNS server to `1.1.1.1` using Netplan or systemd-resolved. Test `ping app.local` and verify that `nslookup google.com` uses the new resolver path. Deliver the configuration and `dns_config.txt`.

Hints: Editing `/etc/hosts` does not require a restart. Global systemd-resolved configuration is in `/etc/systemd/resolved.conf`; after an appropriate configuration change, restart `systemd-resolved`.

Editorial note: `/etc/hosts` is local name resolution, not an authoritative DNS zone. A local stub may appear in nslookup while forwarding to the configured upstream resolver. `.local` is reserved for multicast DNS and is retained here to match the source exercise.

Task 4.2 - Routing Analysis (5 points)

Display the complete routing table, trace the path to `8.8.8.8`, report every observed hop and its latency, and explain in two or three sentences what a router does. Save `routing_analysis.txt`.

Hints: `ip route show` or `route -n`; `traceroute 8.8.8.8` or `mtr 8.8.8.8`; install traceroute if needed with `sudo apt install traceroute`. Use `ip route show table all` when policy-routing tables are relevant.

Task 4.3 - Network Topology (5 points)

Create `network_topology.txt` or a diagram describing this hypothetical company network:

• One router/default gateway: `192.168.1.1`.

• One switch.

• Web, database, and cache servers: `192.168.1.10`, `.11`, and `.12`.

• Ten user computers, described in the source as the range `192.168.1.100-110`.

Draw the connections, using ASCII if desired, and explain what happens if the switch fails.

```
        Internet
            |
         [Router]
            |
         [Switch]
         /  |  \
       Web DB Cache
```

Hints: Common topologies include star, bus, ring, and mesh. This example is a star because devices connect to the switch.

Editorial note: The inclusive range `.100-.110` has eleven addresses. For exactly ten computers, allocate `.100-.109` and reserve `.110`.

Lab 5 - Protocols and Performance (10 bonus points)

Task 5.1 - TCP versus UDP (5 points)

Create `tcp_vs_udp.md` with at least five major differences. Choose and justify a transport for video streaming, FTP file transfer, DNS queries, HTTP/HTTPS browsing, VoIP, and database queries. Use netstat to show the protocols currently used on your host.

Original hints: TCP is connection-oriented and reliable; UDP is connectionless and has no built-in delivery guarantee. The original also labels TCP "slow" and UDP "fast"; explain why that simplification does not determine actual application performance. `netstat -tuln` uses `t` for TCP and `u` for UDP.

Task 5.2 - Network Performance Testing (5 points)

Write `network_performance.sh` to test throughput between two servers with iperf3, measure latency and jitter, calculate packet loss, and save `performance_report.txt`.

Hints:

```
sudo apt install iperf3
iperf3 -s
iperf3 -c <server-ip>
iperf3 -c <server-ip> -u -b 100M
```

If two servers are unavailable, use localhost and explain that it does not measure a physical network link.

Lab 6 - SSL/TLS and HTTPS (15 points)

The company wants to protect its website with HTTPS.

Task 6.1 - A Self-Signed Certificate (5 points)

Generate a private key and a self-signed certificate for `myapp.local` using OpenSSL. Inspect the validity period, subject, and issuer. Save `myapp.key`, `myapp.crt`, and a command/explanation report named `ssl_generation.txt`. The original task says `scenario_1/certs/`; its delivery tree uses `scenario_1_SSL/certs/`. This collection uses the Lab 6 directory.

Original hints:

```
openssl req -x509 -newkey rsa:2048 -nodes \
  -keyout myapp.key -out myapp.crt -days 365 \
  -subj "/C=IR/ST=Tehran/L=Tehran/O=MyCompany/CN=myapp.local"
openssl x509 -in myapp.crt -text -noout
openssl rsa -in myapp.key -check
```

Editorial note: Add a SAN for hostname validation and use `-noout` for a private-key check. Generate the key locally; do not publish it in a public repository.

Task 6.2 - Testing HTTPS with curl (3 points)

Write `test_https.sh` to request a real HTTPS website such as `https://google.com`, inspect its certificate chain, display the negotiated TLS version, and check the expiration date. Save `https_test_report.txt`.

Original hints:

```
curl -vI https://google.com 2>&1 | grep -E "SSL|TLS|expire"
curl --insecure -v https://expired.badssl.com 2>&1
openssl s_client -connect google.com:443 -showcerts
echo | openssl s_client -connect google.com:443 2>/dev/null | \
  openssl x509 -noout -dates
```

Editorial note: Supply SNI and hostname verification when inspecting a named service. `--insecure` disables verification and is only a controlled comparison here. A timeout is not a certificate-validation error.

Task 6.3 - A Python HTTPS Server (4 points)

Write `https_server.py` using a Python HTTP server protected by TLS. Use the certificate from Task 6.1, listen on port 8443, and test with curl. Save the commands and actual results in `https_server_test.txt`.

Original hint:

```
import http.server, ssl
server_address = ('localhost', 8443)
httpd = http.server.HTTPServer(
    server_address, http.server.SimpleHTTPRequestHandler)
httpd.socket = ssl.wrap_socket(
    httpd.socket, server_side=True,
    certfile='myapp.crt', keyfile='myapp.key',
    ssl_version=ssl.PROTOCOL_TLS)
print("Server running on https://localhost:8443")
httpd.serve_forever()
```

Test with `curl -k https://localhost:8443` for the source's insecure local demonstration.

Editorial note: `ssl.wrap_socket` was removed in Python 3.12. The provided solution uses `SSLContext.wrap_socket`. Serve a dedicated public directory so the private key cannot be downloaded. Prefer explicit certificate trust and the correct hostname when validating the local service.

Task 6.4 - TLS Best Practices (3 points)

Create `ssl_best_practices.md` explaining SSL versus TLS, self-signed versus CA-signed certificates, five best practices, and why ordinary HTTP is unsafe compared with HTTPS.

Hints: TLS supersedes SSL; CA means Certificate Authority. Consider maintained TLS versions, strong ciphers, and certificate validation. Explain that HTTPS protects the connection rather than guaranteeing that the application has no security flaws.

Lab 7 - FTP and File Transfer (10 points)

The team needs to transfer files between servers.

Task 7.1 - An FTP Server with vsftpd (5 points)

If installing vsftpd is impossible, provide a theoretical answer with documentation. Otherwise write `setup_ftp.sh` to install vsftpd, create a configuration for anonymous access, create a dedicated FTP user, and include start/stop commands. Deliver `setup_ftp.sh`, `vsftpd.conf`, and `ftp_setup_guide.txt`.

Original hints: Install with `sudo apt install vsftpd`; the configuration is `/etc/vsftpd.conf`.

```
anonymous_enable=NO
local_enable=YES
write_enable=YES
chroot_local_user=YES
```

Editorial note: The requirement says anonymous access, but `anonymous_enable=NO` disables it. Document which policy you implement. The supplied local lab solution uses read-only anonymous access and a dedicated account for uploads. Plain FTP is unencrypted.

Task 7.2 - FTP Client Operations (3 points)

Write `ftp_operations.sh` to connect to an FTP server (the source suggests `ftp.ubuntu.com`), list files, download a file, and upload one if permitted. Document commands and explanations in `ftp_operations_guide.txt`.

Hints:

```
ftp ftp.ubuntu.com
ls
get filename
put filename
bye

wget ftp://ftp.ubuntu.com/path/to/file
curl -u username:password ftp://server.com/file.txt
```

Editorial note: Public mirrors normally do not permit uploads. Use an authorized local server for that test. Prompt for a password rather than storing real credentials in scripts or shell history.

Task 7.3 - FTP, SFTP, and SCP (2 points)

Create `file_transfer_comparison.md` explaining differences, advantages, limitations, suitable scenarios, and one command for each method.

Hints: FTP commonly uses control port 21; SFTP and SCP use SSH, commonly port 22. Basic FTP is insecure on a public network. Current OpenSSH scp uses SFTP by default; distinguish the tool from its legacy protocol.

Lab 8 - HTTP and Web Services (10 points)

The company's API needs configuration and testing.

Task 8.1 - A Python or Nginx Web Server (4 points)

Choose one option:

• **A - Python:** Write a simple HTTP server on port 8080 that serves a small HTML page.

• **B - Nginx:** Write a configuration for a static website, a `myapp.local` virtual host, and custom error pages.

Deliver `http_server.py` or `nginx_config.conf`, plus `http_server_guide.txt`.

Original Python hint:

```
import http.server
import socketserver
PORT = 8080
Handler = http.server.SimpleHTTPRequestHandler
with socketserver.TCPServer(("", PORT), Handler) as httpd:
    print(f"Server at http://localhost:{PORT}")
    httpd.serve_forever()
```

Original Nginx hint:

```
server {
    listen 80;
    server_name myapp.local;
    root /var/www/myapp;
    index index.html;
}
```

Task 8.2 - HTTP Methods and API Testing (4 points)

Write `test_http_methods.sh` to test GET, POST, PUT, and DELETE with curl against a public demonstration API such as JSONPlaceholder. Inspect response codes (examples in the source are 200, 404, and 500), display response headers, and save `http_methods_report.txt`.

Hints:

```
curl -X GET https://jsonplaceholder.typicode.com/posts/1
curl -X POST https://jsonplaceholder.typicode.com/posts \
  -H "Content-Type: application/json" \
  -d '{"title":"test","body":"content","userId":1}'
curl -i https://jsonplaceholder.typicode.com/posts/1
curl -o /dev/null -s -w "%{http_code}\n" https://google.com
```

Record actual codes; do not invent a 500 response. JSONPlaceholder simulates writes rather than persisting your changes.

Task 8.3 - HTTP Response Headers (2 points)

Request several real websites and analyze `Content-Type`, `Server`, `Cache-Control`, `X-Frame-Options`, and `Content-Security-Policy`. Explain each header in `http_headers_analysis.txt`.

Hints: `curl -I https://github.com`, `curl -I https://google.com`, and `curl -I https://stackoverflow.com`. Headers serve security or performance purposes, but they may be absent or differ between redirects and final responses.

Lab 9 - DNS, CDNs, and BIND (12 points)

Task 9.1 - DNS Record Types (4 points)

Create `dns_records_explained.md` explaining A, AAAA, CNAME, MX, TXT, NS, and SOA. Give a real use case for each and obtain real examples with dig or nslookup.

Hints: `dig google.com A`, `AAAA`, `MX`, `TXT`, and `NS`. A maps to IPv4, AAAA to IPv6, CNAME is an alias, MX identifies mail exchangers, and TXT is used for information such as SPF and DKIM. Select another domain if a requested record is absent or a query fails; preserve the failed observation.

Task 9.2 - DNS Query Analysis (4 points)

Write `dns_analyzer.sh` to query a list including `google.com`, `github.com`, `stackoverflow.com`, and `linkedin.com`. Extract IP addresses, TTLs, and answering DNS servers, measure response time, and save a table in `dns_analysis_report.txt`.

Hints: `dig +short google.com`, `dig google.com | grep "Query time"`, and `time dig google.com > /dev/null`. Distinguish the answering resolver from an authoritative name server and DNS query time from page-load time.

Task 9.3 - CDN and Caching (2 points)

Create `cdn_explanation.md` explaining what a CDN does, how it can reduce latency, the difference between origin and edge servers, and an observed website/CDN example using curl response headers.

Hints: CDN means Content Delivery Network. The source mentions Cloudflare, Akamai, and AWS CloudFront. Inspect headers such as `X-Cache`, `CF-Cache-Status`, and `X-Served-By`:

```
curl -I https://cdn.jsdelivr.net/npm/jquery@3.6.0/dist/jquery.min.js
```

Interpret headers according to the provider; a missing vendor-specific header does not prove that no CDN is used.

Task 9.4 - A Sample BIND Configuration (2 points; theoretical)

Create `named.conf.local`, `db.example.local`, and `bind_explanation.txt`. Define a zone for `example.local`, A records for `www`, `mail`, and `ftp`, an MX record for mail, and an NS record for the name server. Installation is **not required**.

Original zone hint:

```
$TTL 604800
@ IN SOA ns1.example.local. admin.example.local. (
    3       ; Serial
    604800  ; Refresh
    86400   ; Retry
    2419200 ; Expire
    604800  ; Negative Cache TTL
)
@    IN NS ns1.example.local.
@    IN A 192.168.1.10
www  IN A 192.168.1.10
mail IN A 192.168.1.20
@    IN MX 10 mail.example.local.
```

The hint is partial; include the requested ftp and name-server address records. `.local` remains a lab-only source name with the multicast-DNS caveat.

Lab 10 - SSH and Remote Management (10 points)

Task 10.1 - SSH Key Authentication (3 points)

Generate an RSA 2048-bit SSH key pair, display the public key in the appropriate format, explain passwordless authentication, and write a convenient SSH client profile. Deliver `ssh_keys_guide.txt` and `ssh_config_sample`.

Original hints:

```
ssh-keygen -t rsa -b 2048 -f ~/.ssh/mykey -N ""
ssh-copy-id -i ~/.ssh/mykey.pub user@server

Host myserver
    HostName 192.168.1.10
    User admin
    IdentityFile ~/.ssh/mykey
    Port 22
```

The empty passphrase is the source's lab example. Protect real private keys and avoid overwriting an existing key. Do not publish private keys.

Task 10.2 - SSH Tunnels (4 points)

Create `ssh_tunneling_guide.md` explaining local forwarding, remote forwarding, dynamic forwarding/SOCKS, an example for each, and a real use case for each.

Original hints:

```
ssh -L 8080:localhost:80 user@server
ssh -R 9000:localhost:3000 user@server
ssh -D 1080 user@server
```

Local port 8080 forwards to port 80 from the server's perspective. Remote server port 9000 forwards to port 3000 from the client's perspective. Dynamic forwarding creates a SOCKS proxy on local port 1080. Explain which machine listens and which connects to the destination.

Task 10.3 - SSH Hardening (3 points)

Create `sshd_config_hardened` and `ssh_security_guide.txt`. Explain which settings improve security, why password and root login may be disabled, and how failed login attempts are monitored.

Original hints:

```
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
Port 2222
MaxAuthTries 3

grep "Failed password" /var/log/auth.log
```

Validate the configuration and a separate key-based login before restricting a real SSH service. A different port does not replace secure authentication.

Lab 11 - Monitoring and Logging (8 points)

Task 11.1 - Network Monitoring (3 points)

Write `network_monitor.sh` to show interface statistics every five seconds, calculate upload/download usage, count active connections, identify the five addresses with the most traffic, and append results to `monitoring_log.txt`.

Hints: `ifstat -i eth0 1 5`, `cat /proc/net/dev`, and `ss -ntu | wc -l`. If specialist tools are unavailable, use netstat and `/proc/net/dev`.

Editorial note: Counters are cumulative bytes. Compute differences over an interval for rates. Counting connections does not identify the most traffic by bytes. Exclude output headers when counting sockets and define the scope of active connections.

Task 11.2 - Log Analysis (3 points)

Write `analyze_logs.sh` to find network-related logs under `/var/log`, extract the **10 most recent events**, extract and count IP addresses, identify suspicious patterns such as repeated failed attempts, and save `log_analysis_report.txt`.

Hints:

```
ls -lh /var/log/syslog /var/log/kern.log /var/log/auth.log
grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' /var/log/auth.log | sort | uniq -c
```

If access is unavailable, create a clearly labeled sample log. Explain supported timestamp formats, compressed files, and journal limitations.

Task 11.3 - A Terminal Dashboard (2 points)

Write `dashboard.sh` showing interfaces/IPs, an active connection count, open ports, DNS servers, and the five most recent log entries. Refresh every ten seconds with watch or a loop. Deliver the script and `dashboard_screenshot.txt` describing its output.

Original hints:

```
while true; do
    clear
    echo "=== Network Dashboard ==="
    ip -br addr
    echo "Connections: $(ss -ntu | wc -l)"
    sleep 10
done
echo -e "\e[32mGreen Text\e[0m"
echo -e "\e[31mRed Text\e[0m"
```

Lab 12 - Practical Troubleshooting (5 points)

Task 12.1 - A Website Cannot Be Reached (2 points)

For "a user cannot access example.com", create `troubleshoot_website.md`. List debugging steps in order, commands for each step, possible root causes, and the solution for each cause.

Hints: Check physical connectivity, IP configuration, DNS, routing, firewall, and the application. Explain each result rather than assuming that one failed probe proves the website is down.

Task 12.2 - Slow Network Performance (2 points)

Write `diagnose_slow_network.sh` to test latency to several external servers, inspect packet loss, measure DNS resolution time, check MTU, and identify a possible bottleneck. Save `network_performance_diagnosis.txt`.

Original hints:

```
ping -c 10 8.8.8.8 | grep avg
ip link show | grep mtu
ping -M do -s 1472 8.8.8.8
time dig google.com
```

A failed ICMP/MTU probe can be filtered. Separate observations from unconfirmed diagnoses.

Task 12.3 - SSH Timeouts (1 point)

Create `timeout_troubleshooting.md` for an SSH connection that times out. List possible causes, a diagnostic command for each, proposed solutions, and a script that simulates the problem.

Hints: Firewall blocking, wrong IP/port, routing problems, a down server, or DNS failure. Test with:

```
timeout 5 ssh user@server
telnet server 22
nc -zv -w 5 server 22
```

Keep the simulation confined to a lab. Distinguish timeout, explicit connection refusal, failed name resolution, and authentication rejection.

Appendix A - Submission and Documentation Instructions

The source assignments ask students to upload an archive through their course panel. Use an English title containing the exercise name and the student's first and last names; examples are `AmirMohammadMohammadi_network__01` and `AmirMohammadMohammadi_network__02`. Ask questions in the course's Telegram group.

For the original separate submissions, use `[Surname]_network_01/` and `[Surname]_network_02/` with the corresponding scenario subdirectories. Compress with:

```
zip -r [Surname]_network_01.zip [Surname]_network_01/
zip -r [Surname]_network_02.zip [Surname]_network_02/
```

This unified collection instead uses `solutions/lab-01/` through `solutions/lab-12/`, with a source mapping in the collection manifest. The task filenames listed above identify the expected deliverables. Extra helper scripts and captured reports are retained where supplied. Store generated private keys locally and exclude them from public archives.

Record all commands in `commands_history.txt`. Include screenshots where required. Some commands need sudo. If a package is missing, install it when appropriate and record the installation; otherwise explain the limitation and an alternative. Set a VM's networking to NAT or bridged mode as appropriate. Localhost is accepted for many Assignment 02 tasks; public services, optional Docker (`docker run -d -p 8080:80 nginx`), and documented simulations are alternatives when hands-on execution is unavailable.

The historical deadlines are "by the start of Sunday" for Assignment 01 and "Wednesday next week" for Assignment 02. The original files do not establish calendar dates here. The source encourages students to read hints, manuals, and documentation first, then search for relevant explanations and ask the instructor when needed. Understanding the concepts and documenting the work matter alongside execution.

Appendix B - Original Grading Criteria

Assignment 01: technical correctness 40%; documentation 30%; creativity 15%; code cleanliness/readability 15%.

Assignment 02: correct implementation 35%; documentation 30%; conceptual understanding 20%; code quality/readability/error handling 15%.

Appendix C - Learning Resources Listed in the Source

• https://www.digitalocean.com/community/tutorials/openssl-essentials-working-with-ssl-certificates-private-keys-and-csrs

• https://howhttps.works/

• https://www.cloudflare.com/learning/dns/what-is-dns/

• https://www.digitalocean.com/community/tutorials/how-to-configure-bind-as-a-private-network-dns-server-on-ubuntu-18-04

• https://www.ssh.com/academy/ssh/tunneling

• https://www.ssh-audit.com/

• https://www.tecmint.com/linux-network-monitoring-tools/

• https://www.wireshark.org/docs/

• https://curl.se/docs/httpscripting.html

• https://www.redhat.com/sysadmin/beginners-guide-ssl-certificates

• https://www.cyberciti.biz/faq/

• https://linuxize.com/

These links are retained from the original exercise list, rather than represented as newly verified recommendations. The chapter bibliography provides the technical references used for its explanations.